

iOS App 启动优化：代码实现详解

一、诊断先行：用工具定位问题

代码埋点：测量 main() 到 didFinishLaunching 的耗时

这是一种快速、宏观的评估方法，可以让你立刻知道哪个阶段比较慢。

main.m

Code block

```
1  #import <UIKit/UIKit.h>
2  #import "AppDelegate.h"
3  #import <CoreFoundation/CoreFoundation.h>
4
5  int main(int argc, char * argv[]) {
6      // --- 1. 记录 main 函数开始时间 ---
7      CFAbsoluteTime startTime = CFAbsoluteTimeGetCurrent();
8      // --- 结束 ---
9
10     @autoreleasepool {
11         int result = UIApplicationMain(argc, argv, nil,
12     NSStringFromClass([AppDelegate class]));
13
14         // --- 2. 计算并打印 main 函数耗时 ---
15         CFAbsoluteTime endTime = CFAbsoluteTimeGetCurrent();
16         NSLog(@"main() function took %.2f seconds", endTime - startTime);
17         // --- 结束 ---
18
19         return result;
20     }
```

AppDelegate.m

Code block

```
1  #import "AppDelegate.h"
2  #import <CoreFoundation/CoreFoundation.h>
3
4  @implementation AppDelegate
5
```

```

6  - (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:
    (NSDictionary *)launchOptions {
7
8      // --- 3. 记录 didFinishLaunching 开始时间 ---
9      CFAbsoluteTime startTime = CFAbsoluteTimeGetCurrent();
10     // --- 结束 ---
11
12     // ... 你的初始化代码 ...
13     [self initializeServices];
14
15     // --- 4. 计算并打印 didFinishLaunching 耗时 ---
16     CFAbsoluteTime endTime = CFAbsoluteTimeGetCurrent();
17     NSLog(@"application:didFinishLaunchingWithOptions: took %.2f seconds",
        endTime - startTime);
18     // --- 结束 ---
19
20     return YES;
21 }
22
23 - (void)initializeServices {
24     // 这里是你的一些初始化逻辑
25 }
26
27 @end

```

二、优化实施：三步走战略

第一阶段：优化 dyld 阶段（基础优化）

优化 +load 方法

这是最常见的 dyld 阶段性能杀手。

✗ 错误示范：在 +load 中执行耗时操作

Code block

```

1  // SomeManager.m
2  + (void)load {
3      // ✗ 错误：在 +load 里执行耗时操作
4      [self performExpensiveSetup]; // 比如读取大文件、进行复杂计算
5      [SomeSDK setup]; // 比如初始化一个庞大的 SDK
6  }
7
8  - (void)performExpensiveSetup {

```

```
9      // ... 很慢的代码 ...
10 }
```

✅ 正确示范：将耗时操作移到 `dispatch_once` 或 `+initialize` 中

Code block

```
1  // SomeManager.m
2  + (void)load {
3      // ✅ 优化: +load 只做最轻量事情, 比如注册、钩子等
4      NSLog(@"SomeManager loaded.");
5  }
6
7  + (void)initialize {
8      // 在第一次调用此类的任何方法前执行, 且只执行一次
9      // 如果逻辑复杂, 建议用 dispatch_once
10     static dispatch_once_t onceToken;
11     dispatch_once(&onceToken, ^{
12         [self performExpensiveSetup];
13     });
14 }
15
16 - (void)performExpensiveSetup {
17     // ... 原来的耗时代码 ...
18 }
```

第二阶段：优化 `main()` 阶段（核心优化）

1. 异步初始化：将非 UI 相关的任务挪到后台

❌ 错误示范：在 `didFinishLaunching` 中同步初始化

Code block

```
1  - (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:
    (NSDictionary *)launchOptions {
2      // ❌ 错误: 同步初始化, 阻塞主线程
3      [AnalyticsService configure]; // 统计 SDK
4      [PushService registerDevice]; // 推送 SDK
5      [UserManager loadUserData]; // 加载用户数据
6      [ImageCache preheat]; // 预热图片缓存
7
8      // ... 其他 UI 设置 ...
9  }
```

```
10     return YES;
11 }
```

✓ 正确示范：在 didFinishLaunching 中异步初始化

Code block

```
1  - (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:
    (NSDictionary *)launchOptions {
2
3      // --- 1. 异步初始化非 UI 任务 ---
4      dispatch_async(dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT,
5          0), ^{
6          [AnalyticsService configure];
7          [PushService registerDevice];
8          [UserManager loadUserData];
9          [ImageCache preheat];
10     });
11     // --- 结束 ---
12     // ... UI 设置（保持在主线程） ...
13
14     return YES;
15 }
```

2. 懒加载（Lazy Loading）：只在需要时才创建对象

✗ 错误示范：在 didFinishLaunching 中创建所有对象

Code block

```
1  - (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:
    (NSDictionary *)launchOptions {
2      // ✗ 错误：App 一启动就创建所有 Manager
3      self.someManager = [[SomeManager alloc] init];
4      self.anotherManager = [[AnotherManager alloc] init];
5      self.yetAnotherManager = [[YetAnotherManager alloc] init];
6
7      return YES;
8  }
```

✓ 正确示范：使用懒加载的 Getter 方法

Code block

```
1  // AppDelegate.h
```

```

2  @property (nonatomic, strong) SomeManager *someManager;
3  @property (nonatomic, strong) AnotherManager *anotherManager;
4
5  // AppDelegate.m
6  - (SomeManager *)someManager {
7      if (!_someManager) {
8          _someManager = [[SomeManager alloc] init]; // 第一次用到时才创建
9      }
10     return _someManager;
11 }
12
13 - (AnotherManager *)anotherManager {
14     if (!_anotherManager) {
15         _anotherManager = [[AnotherManager alloc] init];
16     }
17     return _anotherManager;
18 }
19
20 // 在需要时才调用
21 - (void)someMethodThatNeedsTheManager {
22     // 这时候才会真正创建 SomeManager
23     [self.someManager doSomething];
24 }

```

3. 延迟非紧急任务：用 `dispatch_after` 将任务推迟

❌ 错误示范：在 `didFinishLaunching` 中立即执行非紧急任务

Code block

```

1  - (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:
    (NSDictionary *)launchOptions {
2      // ❌ 错误：App 一启动就执行，可能影响首屏渲染
3      [self updateLocalConfig]; // 更新本地配置
4      [self checkForUpdates]; // 检查 App 更新
5      [self preDownloadAssets]; // 预下载资源
6
7      return YES;
8  }

```

✅ 正确示范：延迟执行

Code block

```

1  - (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:
    (NSDictionary *)launchOptions {

```

```

2
3    // --- 1. 延迟执行非紧急任务 ---
4    dispatch_after(dispatch_time(DISPATCH_TIME_NOW, (int64_t)(2.0 *
NSEC_PER_SEC)), dispatch_get_main_queue(), ^{
5        [self updateLocalConfig];
6        [self checkForUpdates];
7        [self preDownloadAssets];
8    });
9    // --- 结束 ---
10
11    return YES;
12 }

```

第三阶段：终极优化：二进制重排（Binary Reordering）

这个是最复杂、效果也最显著的优化，需要借助脚本和编译器参数。

核心思想：让 App 启动时需要的函数，在二进制文件中尽可能地“挨在一起”，这样可以减少内存分页（Page Faults）。

实现步骤

1. **生成函数调用列表：**使用 dtrace 工具跟踪一次 App 的启动过程，记录下所有被调用的函数。

Code block

```

1  # 1. 将以下脚本保存为 startup.d
2  #!/usr/sbin/dtrace -s
3  pidtarget::start_wqthread:entry
4  {
5      self->trace = 1;
6  }
7
8  pidtarget:::return
9  /self->trace/
10 {
11     @[probefunc] = count();
12 }
13
14 END
15 {
16     trunc(@, 1000);
17 }

```

2. **执行跟踪：**在命令行运行以下命令，启动你的 App，然后手动操作几秒钟，最后按 Ctrl+C 结束。

Code block

```
1 sudo dtrace -arch arm64 -s startup.d -c "./build/Release-iphoneos/YourApp.app/YourApp"
```

3. **生成 Order File**：将 dtrace 的输出结果，按照特定格式整理成一个 .order 文件。

Code block

```
1 # Generated by dtrace
2 _main
3 _UIApplicationMain
4 __mh_execute_header
5 -[AppDelegate application:didFinishLaunchingWithOptions:]
6 -[ViewController viewDidLoad]
7 # ... 更多函数名 ...
```

4. **配置 Xcode**：在 Xcode 的 Build Settings 中，找到 Order File 选项，将你生成的 .order 文件路径填入。

通过以上四个步骤，Xcode 在最终链接（Link）App 时，就会参考这个 .order 文件，将函数按顺序排列，从而优化启动时的内存加载效率。

三、优秀的 App 启动优化第三方工具

这些工具的核心功能是性能监控和分析，它们能帮你自动发现启动慢、卡顿等问题，并给出详细的报告。

工具名称	语言	核心功能	特点	推荐指数
Firebase Performance Monitoring	多平台	应用性能监控	Google 官方出品，可以自动监测 App 启动时间、网络请求、屏幕渲染等性能指标，并提供详细报告。与 Crashlytics 等服务集成良好。	★★★★★
腾讯 Bugly	iOS/Android	崩溃监控 + 性能监控	除了崩溃，Bugly 也提供了 ANR、卡顿、启动耗时等性能数据监控，是国内开发者常用的选择。	★★★★★

网易链家 Matrix	iOS/Android	APM 应用性能管理系统	开源，功能强大，对启动时间、内存、卡顿、网络等都有深入的分析能力，适合有技术实力的团队自研和二次开发。	★★★★★★
Hertz	iOS	启动时间分析	一个专门用于 iOS App 启动时间分析的开源工具。它通过 Hook objc_msgSend 等关键函数，生成可视化的火焰图（Flame Graph），让你能非常直观地看到哪个函数调用最耗时。	★★★★★
Chisel	iOS	LLDB 调试辅助工具	一套 LLDB 命令集合，其中 pvc、fv 等命令在调试 UI 和性能问题时非常高效，可以作为开发时的辅助工具。	★★★

如何选择？

如果你已经在使用 Firebase 或 Bugly：那么它们提供的性能监控功能是首选。它们能将性能数据与崩溃数据关联起来，形成更完整的分析闭环。例如，你可能会发现某个版本启动时间变长，同时崩溃率也上升了，这很可能意味着新引入的代码既慢又不稳定。

总结

这些代码示例从“快速诊断”到“核心优化”再到“终极优化”，构成了一个完整的 App 启动优化实践方案。记住，优化是一个持续的过程，需要不断地测量、分析、优化、验证。

（注：文档部分内容可能由 AI 生成）